

Pocket SDR ver. 0.14

Algorithms and Models

2026-03-16

Contents

1. System Overview	1
2. IF Data Handling	2
3. Spreading Code Generation	3
4. Signal Acquisition	4
5. Signal Tracking	5
6. Navigation Data Decoding	7
7. Error Correction	8
8. PVT Generation	9
9. Snapshot Positioning	10
10. Receiver Channel State Machine	11
References	12

1. System Overview

Pocket SDR is an open-source GNSS software-defined radio (SDR) receiver. It processes digitized intermediate-frequency (IF) samples from the Pocket SDR FE hardware frontend and produces GNSS signal acquisition results (code offset, Doppler, C/N0), tracked signal observables (carrier phase, pseudorange, Doppler), decoded navigation data (ephemerides, time, corrections), and Position/Velocity/Time (PVT) solutions.

The software is organized into a layered pipeline:

```
RF Front-end (Pocket SDR FE)
| digitized IF samples (int8)
v
IF Data Buffer
|
+---> Acquisition (parallel code search)
|
+---> Tracking --> Nav Decoding --> PVT
```

Supported GNSS Systems and Signals

System	Signals
GPS	L1C/A, L1C-D, L1C-P, L2C-M, L5-I, L5-Q
GLONASS	L1C/A (FDMA), L2C/A (FDMA), L1OCd, L1OCp, L2OCp, L3OCd, L3OCp
Galileo	E1-B, E1-C, E5a-I, E5a-Q, E5b-I, E5b-Q, E6-B, E6-C
QZSS	L1C/A, L1C/B, L1C-D, L1C-P, L1S, L2C-M, L5-I, L5-Q, L5S-I, L5S-Q, L6D, L6E
BeiDou	B1I, B1C-D, B1C-P, B2a-D, B2a-P, B2I, B2b-I, B3I
NavIC	L1-SPS-D, L1-SPS-P, L5-SPS
SBAS	L1C/A, L5-I, L5-Q

2. IF Data Handling

2.1 Data Format

The Pocket SDR FE captures IF data in one of the following formats:

Format	Description
I-sampling (real)	Single int8 sample per time step
IQ-sampling (complex)	Interleaved int8 pairs: I, Q, I, Q, ...

The `read_data()` function (`sdr_func.py`) reads raw int8 samples from a binary file and converts them to complex64 (numpy complex64) arrays. For I-sampling: `data = int8_sample + 0j`. For IQ-sampling: `data = I_sample - j * Q_sample`. The negative sign on Q derives from the down-conversion convention used by the MAX2771 frontend IC.

2.2 Sampling Parameters

Key parameters for IF data processing:

Parameter	Symbol	Typical Values
Sampling frequency	<code>fs</code>	12 MHz, 24 MHz, 48 MHz
IF frequency	<code>fi</code>	0 Hz (zero-IF) or non-zero
Sample type	IQ	1 (I-only) or 2 (IQ)

For zero-IF (`fi = 0`), IQ-sampling is assumed automatically.

2.3 GLONASS FDMA Frequency Shift

GLONASS uses FDMA with per-satellite frequency channel numbers (FCN). The IF frequency is shifted accordingly before correlation:

G1CA: $fi_shifted = fi + 0.5625 \text{ MHz} \times FCN$

G2CA: $fi_shifted = fi + 0.4375 \text{ MHz} \times FCN$

3. Spreading Code Generation

3.1 Code Types

Pocket SDR generates spreading codes for all supported signals in software. The primary code is a PRN-specific binary sequence (chip values ± 1). Several signal types additionally use a secondary (overlay) code modulated on top of the primary code to extend coherent integration.

Signal	Primary Code	Secondary Code
GPS L1C/A	1023-chip Gold code	None
GPS L1C-D	10230-chip Weil code	1800-chip overlay
GPS L5-I	10230-chip XB code	10-chip Neumann-Hofman
Galileo E1-B	4092-chip memory code	None
Galileo E5a-I	10230-chip memory code	20-chip secondary
QZSS L6D/L6E	10230-chip CSK code	None
BeiDou B1C-D	10230-chip Weil code	1800-chip overlay
GLONASS L1C/A	511-chip m-sequence	None

3.2 Code Generation Method

Most codes are generated by linear feedback shift registers (LFSRs) or stored memory tables derived from the respective ICD. The generation polynomial for each signal follows its official interface control document (ICD). Code generation for all supported signals is implemented in `sdr_code.py` and `sdr_code_gal.py`.

3.3 Code FFT Pre-computation

For the FFT-based correlator, the code DFT (discrete Fourier transform) is pre-computed once per channel and reused across all Doppler bins:

```
code_fft = conj( FFT( code_resampled_with_zero_padding ) )
```

Zero-padding to twice the code length ($2N$) is optional (the `-nz` flag disables it). Zero-padding enables circular correlation without aliasing.

3.4 Code Resampling

When the code chip rate does not divide evenly into the sampling frequency, the code is resampled to the sample grid. For tracking, a code bank of `N_CODE = 10` versions with sub-sample offsets is maintained to allow fractional-chip code offset tracking without re-interpolation at every epoch.

4. Signal Acquisition

4.1 Parallel Code Search

Signal acquisition searches a two-dimensional space of code offset τ and Doppler frequency f_d . The search uses a parallel code search algorithm based on the circular cross-correlation property of the FFT.

For each Doppler bin f_{d_i} :

```
1. Mix IF data with a local carrier at  $f_i + f_{d_i}$ :
data_mixed[n] = data[n] * exp(-j 2pi (f_i + f_{d_i}) n / f_s)

2. Compute circular cross-correlation via FFT:
C[tau] = IFFT( FFT(data_mixed) * code_fft )
where code_fft = conj( FFT(code) )

3. Compute correlation power:
P[i, tau] = |C[tau]|^2
```

The result is a 2D power surface $P(f_d, \tau)$.

4.2 Non-Coherent Integration

To improve sensitivity, correlation powers from multiple code cycles are accumulated non-coherently (power summation, not complex summation):

```
P_sum += P[i, tau] for each code cycle
```

The default non-coherent integration time is $T_{ACQ} = 10$ ms (approximately 10 code cycles for 1 ms codes). This avoids the navigation data bit ambiguity while providing approximately 10 dB integration gain.

4.3 Detection and Threshold

After integration, the peak is found and the C/N_0 metric is computed:

```
P_max = max(P_sum)
P_ave = mean(P_sum)
C/N0 = 10 * log10( (P_max - P_ave) / P_ave / T_code ) [dB-Hz]
```

The signal is declared acquired when $C/N_0 \geq 35$ dB-Hz.

4.4 Fine Doppler Estimation

The coarse Doppler bin index is refined by fitting a quadratic polynomial to the three power values around the peak bin:

```
p(f) = a*f^2 + b*f + c
f_fine = -b / (2*a)
```

This provides sub-bin Doppler accuracy without reducing the bin spacing.

4.5 Doppler Search Range

Default Doppler search range: ± 5000 Hz. Search step: $DOP_STEP / T_code = 0.5 / T_code$ Hz (0.5 bins per code cycle). For GLONASS FDMA signals, the search centre is shifted by the FCN frequency offset before the search.

5. Signal Tracking

5.1 Correlator Structure

The tracking loop uses a set of correlators at different code phase offsets:

Correlator	Offset	Purpose
Prompt (P)	0 chips	Phase/frequency error discrimination
Early (E)	+d/2 chips	DLL discriminator
Late (L)	-d/2 chips	DLL discriminator
Noise (N)	-80 samples	Noise floor estimate for C/N0

Default correlator spacing: $d = 0.25$ chips (SP_CORR). At each epoch (one code cycle), the correlator outputs are computed by mixing the IF data with the local carrier replica and integrating against the local code replica:

$$C_k = \text{SUM}\{ \text{data}[n] * \exp(-j \, 2\pi \, f_{c_k} \, n / f_s) * \text{code}[n - \text{tau}_k] \} / N$$

For QZSS L6D/L6E (CSK modulation), the FFT correlator is used instead (see Section 5.6).

5.2 Carrier Tracking: FLL and PLL

The tracking loop uses a two-stage approach:

Stage 1 - Frequency Lock Loop (FLL): Active for the first $T_{\text{FPULLIN}} = 1.0$ s to achieve initial frequency lock.

The FLL discriminator uses the cross/dot product of consecutive P correlator outputs:

$$\begin{aligned} \text{dot} &= I1 \cdot I2 + Q1 \cdot Q2 \\ \text{cross} &= I1 \cdot Q2 - Q1 \cdot I2 \\ \text{err_freq} &= \text{atan}(\text{cross} / \text{dot}) \text{ [Costas, for BPSK]} \\ &= \text{atan2}(\text{cross}, \text{dot}) \text{ [non-Costas, for data-less pilot]} \end{aligned}$$

Frequency update:

$$f_d = (B_{\text{FLL}} / 0.25) * \text{err_freq} / (2 * \pi)$$

FLL bandwidth transitions from 5 Hz (wide) to 2 Hz (narrow) at the halfway point of the pull-in time.

Stage 2 - Phase Lock Loop (PLL): Activated after FLL pull-in. Uses a second-order PLL ($B_{\text{PLL}} = 5.0$ Hz):

$$\begin{aligned} \text{err_phas} &= \text{atan}(Q_P / I_P) / (2 * \pi) \text{ [Costas]} \\ &= \text{atan2}(Q_P, I_P) / (2 * \pi) \text{ [non-Costas]} \\ W &= B_{\text{PLL}} / 0.53 \\ f_d &+= 1.4 * W * (\text{err_phas} - \text{err_phas_prev}) + W^2 * \text{err_phas} * T \end{aligned}$$

5.3 Code Tracking: DLL

A non-coherent Early-minus-Late (EML) delay lock loop (DLL) tracks the code phase. The DLL runs every $N = T_{\text{DLL}} / T_{\text{code}}$ epochs (default: 10 epochs for 1 ms codes) using accumulated E and L magnitudes.

Discriminator (normalized):

$$\text{err_code} = (|E| - |L|) / (|E| + |L|) / 2 * T_{\text{code}} / \text{code_length} \text{ [s]}$$

Code phase update:

$$\text{coff} = (B_{\text{DLL}} / 0.25) * \text{err_code} * T_{\text{code}} * N \text{ [} B_{\text{DLL}} = 0.25 \text{ Hz]}$$

Carrier-aided code tracking:

$$\text{coff} = f_d / f_c * dt$$

Carrier-aided code tracking reduces code tracking noise by incorporating the tracked carrier Doppler, where f_c is the carrier frequency and dt is the elapsed time since the last update.

5.4 C/N0 Estimation

C/N0 is estimated using the signal-to-noise ratio of the Prompt correlator output versus the noise reference correlator (N):

$$C/N_0 = 10 * \log_{10} (\text{SUM}|P|^2 / \text{SUM}|N|^2 / T_{\text{code}}) \text{ [dB-Hz]}$$

The estimate is smoothed with an exponential moving average ($\alpha = 0.5$) over $T_{\text{CN0}} = 1.0$ s. Signal lock is declared lost when $C/N_0 < 32$ dB-Hz.

5.5 Secondary Code Synchronization

Pilot signals (e.g., GPS L1C-P, Galileo E1-C, BeiDou B1C-P) modulate the primary code with a secondary (overlay) code. Pocket SDR detects and removes this secondary code by computing the cross-correlation of the P correlator history with the known secondary code sequence. Synchronization is declared when the normalized correlation exceeds $\text{THRES_SYNC} = 0.02$. After synchronization, each P correlator output is multiplied by the corresponding secondary code chip.

5.6 CSK Demodulation (QZSS L6)

QZSS L6D/L6E uses Code-Shift Keying (CSK) modulation. The transmitted symbol is encoded as a circular shift of the spreading code. Pocket SDR decodes it using the following procedure:

1. Compute full circular cross-correlation over the code period via FFT.
2. Interpolate the correlation power profile at ± 255 chips around center.
3. Detect the peak position; the peak shift index gives the CSK symbol (0-255).
4. The correlator output at the peak position is used as the P correlator.

6. Navigation Data Decoding

6.1 Symbol Synchronization

Navigation data bits (symbols) are extracted from the P correlator outputs. Symbol boundaries are detected by monitoring transitions in the sign of accumulated P correlator outputs. Once synchronized, each bit is decided by the sign of the integrated P output over one symbol period.

6.2 Frame Synchronization

Each navigation message format has a known preamble or sync word. Pocket SDR searches for the preamble in the accumulated symbol buffer. On detection, the frame boundary and data polarity (normal/inverted) are established.

6.3 Supported Navigation Messages

Signal	Message Format	Key Content
GPS L1C/A	LNAV (25 words x 30 bits)	Ephemeris, almanac, UTC/iono
GPS L1C-D	CNAV-2 (subframe 2 & 3)	Ephemeris
GPS L2C-M	CNAV	Ephemeris, clock corrections
GPS L5-I	CNAV	Ephemeris, clock corrections
Galileo E1-B	I/NAV (page pairs)	Ephemeris, iono, UTC
Galileo E5b-I	I/NAV	Ephemeris
Galileo E6-B	HAS (High Accuracy Service)	PPP corrections
QZSS L1C/A	LNAV	Ephemeris (QZSS-specific)
QZSS L6D	CLAS message (Reed-Solomon)	Centimetre-level augmentation
BeiDou B1I	D1/D2 NAV	Ephemeris, almanac
BeiDou B1C-D	CNAV-1	Ephemeris
BeiDou B2a-D	CNAV-2	Ephemeris
GLONASS L1C/A	GLONASS NAV (strings)	Ephemeris, almanac
NavIC L5-SPS	IRNSS NAV	Ephemeris

7. Error Correction

7.1 Convolutional Coding / Viterbi Decoding

Many GNSS navigation messages (GPS L5, Galileo E1-B/E5, BeiDou B1C) use rate-1/2, constraint-length-7 convolutional channel coding. Pocket SDR decodes these using the Viterbi algorithm implemented in the libfec external library.

Parameter	Value
Code rate	1/2
Constraint length	K = 7
Generator polynomial G1	0x4F
Generator polynomial G2	0x6D

7.2 CRC Checking

Navigation frames include CRC (cyclic redundancy check) fields that are verified after Viterbi decoding. Only frames passing CRC are accepted.

7.3 LDPC Decoding (BeiDou B1C, B2a, B2b)

BeiDou new-generation signals use Low-Density Parity-Check (LDPC) codes. Pocket SDR provides two decoders:

Full LDPC decoder (sdr_ldpc.py):

Uses the LDPC-codes external library (sum-product / belief propagation).

NB-LDPC decoder (sdr_nb_ldpc.py):

Compact non-binary LDPC decoder optimized for B1C, B2a, and B2b subframes.

7.4 BCH Error Correction (BeiDou B1I)

BeiDou B1I uses BCH(15, 11, 1) error correction for certain fields. Pocket SDR implements this via a pre-computed error correction lookup table (BCH_CORR_TBL in sdr_nav.py) covering all single-bit error patterns.

7.5 Reed-Solomon Decoding (QZSS L6)

QZSS L6D CLAS data uses Reed-Solomon error correction applied across message frames. Pocket SDR decodes this using the libfec library's RS(255, 223) implementation.

8. PVT Generation

8.1 Overview

The PVT module (`sdr_pvt.c`) generates position, velocity, and time solutions using RTKLIB as the positioning engine. PVT is computed at 1 Hz epochs.

8.2 Observation Data

At each epoch, observables are assembled from all locked channels:

```
Pseudorange : derived from tracked code phase offset and receiver time
Carrier phase: integrated from accumulated Doppler and current carrier phase
Doppler      : from the tracked carrier frequency offset (fd)
C/N0         : from the C/N0 estimator (Section 5.4)
```

An elevation mask of 15 degrees is applied to reject low-elevation satellites.

8.3 Positioning Engine

PVT uses RTKLIB's least-squares single-point positioning algorithm. The solution accounts for:

- Satellite positions and velocities from decoded ephemerides
- Ionospheric delay (Klobuchar model for GPS/QZSS, Galileo NeQuick broadcast)
- Tropospheric delay (Saastamoinen model)
- Satellite clock corrections from navigation data
- Relativistic corrections

8.4 Output Formats

Format	Description
NMEA 0183	GGA, RMC, GSA, GSV sentences (standard marine/mapping format)
RTCM3	RTCM 10403.x binary format for differential corrections
Internal log	CSV-format \$POS, \$OBS, \$SAT log records

9. Snapshot Positioning

9.1 Principle

Snapshot positioning (`pocket_snap.py`, `pocket_snap`) computes a position from a single short capture of IF data (typically a few milliseconds), without requiring continuous tracking or navigation data download. It uses a coarse time and approximate position to predict the expected code offset for each visible satellite, acquire signals to measure the actual code offsets, and compute a position from the pseudorange differences.

9.2 Algorithm

1. Signal acquisition:

Run parallel code search over the snapshot buffer
for each candidate satellite and signal.

2. Pseudorange prediction:

Using the known approximate receiver position and time,
compute predicted pseudoranges from broadcast ephemerides.

3. Residual computation:

Compute pseudorange residuals between measured and predicted values.

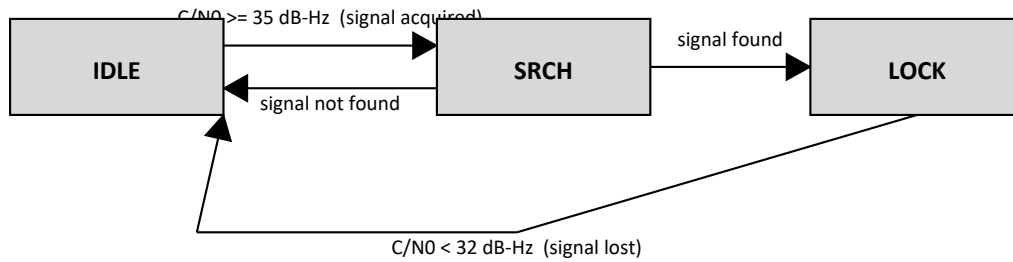
4. Position solution:

Solve for position using RTKLIB's positioning engine
with the residual pseudoranges.

This approach requires an initial position estimate (within approximately 150 km) and approximate time (within approximately 1 minute).

10. Receiver Channel State Machine

Each receiver channel (`sdr_ch.py`) is an independent state machine with three states:



State	Description
IDLE	Waiting for the next acquisition trigger
SRCH	Running parallel code search (acquisition)
LOCK	Running tracking loops (DLL/FLL/PLL) and decoding navigation data

10.1 State Transitions

IDLE -> SRCH : Triggered externally by the receiver scheduler.

SRCH -> LOCK : Signal found ($C/N_0 \geq 35$ dB-Hz after T_{ACQ} integration).

SRCH -> IDLE : Signal not found after T_{ACQ} integration.

LOCK -> IDLE : C/N_0 drops below 32 dB-Hz (signal lost).

10.2 Timing Parameters

Parameter	Symbol	Default	Description
Acquisition integration	T_{ACQ}	10 ms	Non-coherent integration for acquisition
FLL pull-in	$T_{FPULLIN}$	1.0 s	Duration of FLL before switching to PLL
Nav pull-in	$T_{NPULLIN}$	1.5 s	Delay before starting nav data decoding
DLL integration	T_{DLL}	10 ms	Non-coherent integration for DLL
C/N0 averaging	T_{CNO}	1.0 s	Averaging window for C/N0 estimator
Lock threshold	$THRES_CNO[0]$	35 dB-Hz	C/N0 to declare acquisition success
Lost threshold	$THRES_CNO[1]$	32 dB-Hz	C/N0 below which tracking is lost

References

- [1] IS-GPS-200K, NAVSTAR GPS Space Segment/Navigation User Segment Interfaces, May 19, 2019.
- [2] IS-GPS-800F, Navstar GPS Space Segment / User Segment L1C Interfaces, March 4, 2019.
- [3] IS-GPS-705A, Navstar GPS Space Segment / User Segment L5 Interfaces, June 8, 2010.
- [4] Galileo Open Service Signal In Space Interface Control Document, Issue 2.0.
- [5] Galileo E6-B/C Codes Technical Note, Issue 1, January 2019.
- [6] IS-QZSS-PNT-004, Quasi-Zenith Satellite System Interface Specification, November 5, 2018.
- [7] IS-QZSS-L6-003, Quasi-Zenith Satellite System Interface Specification - Centimeter Level Augmentation Service, August 20, 2020.
- [8] BeiDou Navigation Satellite System Signal In Space ICDs (B1I v3.0, B1C v1.0, B2a v1.0, B2b v1.0, B3I v1.0).
- [9] GLONASS Interface Control Document (L1, L2, L3 bands).
- [10] IRNSS/NavIC SIS ICD for Standard Positioning Service, version 1.1, August 2017.
- [11] NavIC Signal in Space ICD for Standard Positioning Service in L1, version 1.0, August 2023.
- [12] RTKLIB: An Open Source Program Package for GNSS Positioning (<https://rtklib.com/>).
- [13] libfec: A library for Forward Error Correction (<https://github.com/quiet/libfec>).
- [14] LDPC-codes: A library for LDPC decoding (<https://github.com/radfordneal/LDPC-codes>).
- [15] T.Takasu, Pocket SDR: Design, Implementation and Applications, A seminar for GNSS Software Defined Receivers, Nov 19, 2024.